



Universidade do Minho
Escola de Letras, Artes e Ciências Humanas

Relatório do Projeto Final de FPLN

Trabalho desenvolvido por:

João Garção (PG 56354)

Joana Azevedo (PG 56352)

Introdução

Este corpus constitui o relatório do projeto final da U.C (Unidade Curricular) de **FPLN** (Fundamentos e Processamento de Linguagem Natural), U.C do primeiro ano do mestrado em Humanidades Digitais, na Universidade do Minho. Assim, foi exigida aos alunos a criação de uma interface python passível de analisar um website e devolver um conjunto de informações previamente definidas pela docente. Para a realização deste projeto, utilizou-se um website de receitas, neste caso o do 24 Kitchen, e optou-se por escolher 6 receitas para analisar, as quais documentamos numa pasta em formato .txt. Também definimos como indispensável para a realização deste projeto o professor José João, o qual nos auxiliou com alguns problemas que encontrámos na produção do código python, assim como apresentar um ponto de vista sobre o projeto.

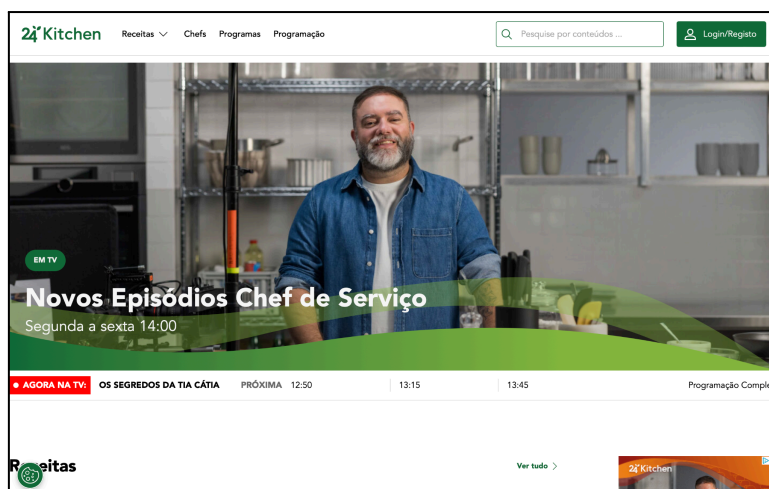


Fig. 1- Página inicial do website do 24 Kitchen.

Desenvolvimento

Imports Utilizados:

```
import os
import re
import csv
import json
import requests
from bs4 import BeautifulSoup
from collections import Counter
import xml.etree.ElementTree as ET
```

Explicação dos Imports Utilizados:

1. **import os**

- Função: Permite interagir com o sistema de arquivos, como navegar através de diretórios, elencar arquivos ou construir caminhos de forma multiplataforma.

- Aplicação no projeto:

- É usado na função para elencar os arquivos num diretório (`os.listdir`) e construir o caminho completo para cada arquivo (`os.path.join`).

2. **import re**

- Função: Apoia expressões regulares que permitem procurar padrões em strings de forma eficiente.

- Aplicação no projeto:

- É utilizado para localizar e extrair informações específicas dos arquivos, como o **autor**, os **ingredientes** e a **preparação**.

3. **import csv**

- Função: Facilita a leitura e escrita de arquivos no formato CSV¹ (Comma-Separated Values).

- Aplicação no projeto:

- Caso haja necessidade de exportar os dados extraídos para um arquivo CSV (esta funcionalidade é opcional e depende do utilizador da interface), pode ser usado para criar um relatório tabular das receitas.

4. **import json**

- Função: Permite utilizar dados no formato JSON², incluindo para a leitura e escrita de arquivos.

- Aplicação no projeto:

- Poder ser usado para guardar os dados extraídos num arquivo JSON, o que permite um armazenamento estruturado e reutilizável.

5. **import requests**

- Função: Permite fazer solicitações HTTP³ para interagir com API's ou aceder a conteúdos da web.

- Aplicação no projeto:

- Pode ser utilizado para fazer o download de receitas ou informações de um site, caso o utilizador procure dados online.

¹ Um ficheiro CSV (valores separados por vírgulas) é um ficheiro de texto com um formato específico que permite que os dados sejam guardados num formato estruturado de tabela.

² O JSON é um formato que armazena informações estruturadas e é principalmente usado para transferir dados entre um servidor e um cliente. O arquivo é basicamente uma alternativa simples e mais leve ao XML (Extensive Markup Language), que tem funções similares.

³ HTTP, ou Hypertext Transfer Protocol, é um protocolo usado para buscar recursos, como arquivos HTML. O HTTP fornece um padrão de mensagens para facilitar trocas de comunicação entre clientes da Web (por exemplo, um navegador) e servidores da Web.

⁴ API significa Application Programming Interface (Interface de Programação de Aplicação). No contexto de APIs, a palavra Aplicação refere-se a qualquer software com uma função distinta. A interface pode ser pensada como um contrato de serviço entre duas aplicações.

6. `from bs4 import BeautifulSoup`

- Função: Este import é usado para extrair informações de documentos HTML ou XML⁵. Facilita o "web scraping".
- Aplicação no projeto:
 - Pode ser usado para extrair receitas diretamente de páginas web, estruturando o conteúdo retirado.

7. `from collections import Counter`

- Função: Oferece uma maneira fácil de contar objetos hasháveis⁶, como palavras ou itens em uma lista.
- Aplicação no projeto:
 - Pode ser usado para identificar os ingredientes mais comuns nas receitas processadas ou para fazer análises estatísticas das mesmas.

8. `import xml.etree.ElementTree as ET`

- Função: Fornece uma API para criar, manipular e analisar documentos XML.
 - Aplicação no projeto:
 - Caso os dados das receitas sejam fornecidos em formato XML, esta biblioteca pode ser usada para processá-los.
-

Tarefa 1

Como tarefa inicial proposta pela docente, foi-nos incumbida a tarefa de escolher uma fonte para recolher os nossos textos para posteriormente os analisar. Como tal, esta tarefa constitui a parte embrionária do projeto, na qual escolhemos o website para analisar, assim como as receitas. O website escolhido foi, como dito anteriormente, o website 24 Kitchen (<https://www.24kitchen.pt/>) e as receitas escolhidas foram as seguintes:

- Bife à portuguesa;
- Bolo de cenoura;
- Caril de castanhas;
- Creme de lagostim;
- Margarita de morango e hortelã;

⁵ A Extensible Markup Language (XML) é uma linguagem de marcação que fornece regras para definir quaisquer dados. Ao contrário de outras linguagens de programação, a XML não pode realizar operações de computação por contra própria.

⁶ Hashable, ou hashável em português, é uma interface que permite que objetos sejam usados como chaves (keys).

- Rabanadas com calda de vinho do Porto;

Após selecionar as receitas anteriores e de as passar para o formato .txt apenas com as informações que achamos mais relevantes, procedeu-se a guardá-las numa pasta, de modo a que futuramente o programa python as pudesse aceder.

Informações relevantes que procuramos extrair:

- Nome da receita;
- Autor da receita (chef que criou a receita);
- Ingredientes (ingredientes necessários para a preparação da receita);
- Preparação (modo de preparar a receita de forma a deixá-la pronta a comer);

```
def extrair_informacoes_pasta(caminho_pasta):

    dados_arquivos = []

    for arquivo in os.listdir(caminho_pasta):

        if arquivo.endswith(".txt"): # Processar apenas arquivos .txt

            caminho_arquivo = os.path.join(caminho_pasta, arquivo)

            with open(caminho_arquivo, "r", encoding="utf-8") as f: # Ler o conteúdo do
arquivo

                linhas = f.readlines() # Ler todas as linhas do arquivo

                titulo = linhas[0].strip() if linhas else "Título não encontrado" # Usar a
primeira linha como título

                conteudo = "".join(linhas) # Unir todas as linhas para a procura de autor,
ingredientes e preparação

                autor = re.search(r"Receita de[:\~]?s*(.+)", conteudo, re.IGNORECASE) #
Extrair o autor da Receita

                autor = autor.group(1).strip() if autor else "Autor não encontrado"
```

```

        ingredientes = re.search(r"Ingredientes[:\-]?s*(.*?) (?:\n\s*Preparação)",
conteudo, re.IGNORECASE | re.DOTALL) # Extrair ingredientes (texto entre "Ingredientes" e
"Preparação")

        if ingredientes:# Se ingredientes foram encontrados, dividir em itens por
linha

            ingredientes_texto = ingredientes.group(1).strip()

            ingredientes_lista = [item.strip() for item in
ingredientes_texto.split("\n") if item.strip()]

        else:

            ingredientes_lista = []

        preparacao = re.search(r"Preparação[:\-]?s*(.*)", conteudo, re.IGNORECASE
| re.DOTALL)# Captura todo o conteúdo após a palavra "Preparação" até o final do arquivo

        if preparacao:# Se preparação foi encontrada, mostarr o conteúdo

            preparacao_texto = preparacao.group(1).strip()

            #preparacao_lista = [item.strip() for item in
preparacao_texto.split("\n") if item.strip()]

            preparacao_lista = preparacao_texto.splitlines()

        else:

            preparacao_lista = []

        dados = { # Armazenar as informações num dicionário

            "titulo": titulo,

            "autor": autor,

            "ingredientes": ingredientes_lista,

            "preparacao": preparacao_lista,

        }

        dados_arquivos.append(dados)

```

```
return dados_arquivos
```

Explicação do código: À medida que se foi redigindo o código, decidiu-se ir comentando as linhas de código de forma a ser mais fácil diferir que blocos de código fazem o quê, assim como destacar a pertinência de cada bloco de código de maneira a que, se algo estiver errado, ser mais fácil saber onde fazer a alteração. Deste modo, temos que:

A função **extrair_informacoes_pasta** processa arquivos de texto armazenados localmente numa pasta e extrai as informações sobre as receitas. Esta função percorre todos os arquivos .txt no diretório especificado e, para cada arquivo, lê o conteúdo para identificar o título, o autor, os ingredientes e as etapas de preparação da receita. Para isso, utiliza expressões regulares que localizam trechos específicos do texto, como o título, o bloco de ingredientes e o bloco de preparação. Os dados extraídos são organizados em dicionários, contendo listas estruturadas para os ingredientes e os passos de preparação, e retornados em uma lista. A função é útil para centralizar e estruturar informações de receitas armazenadas localmente.

Tarefa 2

Para a segunda tarefa, foi-nos exigido escrever um programa capaz de ler os ficheiros gerados na tarefa anterior e extrair a sua informação. Para tal, usámos o seguinte código:

```
# Função para extrair informações de uma URL do 24Kitchen

def extrair_texto_de_url(url, pasta_destino):

    try:

        resposta = requests.get(url)

        resposta.raise_for_status() # Verifica erros na requisição

        sopa = BeautifulSoup(resposta.text, "html.parser")

        elemento_titulo = sopa.find("h2", class_="page-header__title") # Extrair título

        titulo = elemento_titulo.get_text(strip=True) if elemento_titulo else "Título não encontrado"
```

```

elemento_autor = sopa.find("a", class_="page-header__link") # Extrair autor

autor = elemento_autor.get_text(strip=True) if elemento_autor else "Autor não
encontrado"

elemento_titulo_ingredientes = sopa.find("h3", class_="recipe-ingredients__title")
# Extrair ingredientes (título)

titulo_ingredientes = elemento_titulo_ingredientes.get_text(strip=True) if
elemento_titulo_ingredientes else "Título Ingredientes não encontrado"

elementos_ingredientes = sopa.find_all("li", class_="recipe-ingredients__item") #
Extrair ingredientes (lista)

ingredientes = [item.get_text(strip=True) for item in elementos_ingredientes] if
elementos_ingredientes else ["Ingredientes não encontrados"]

elemento_titulo_preparacao = sopa.find("h3", class_="recipe-steps__title") #
Extrair preparação (título)

titulo_preparacao = elemento_titulo_preparacao.get_text(strip=True) if
elemento_titulo_preparacao else "Título Preparação não encontrado"

elementos_preparacao = sopa.find_all("li", class_="recipe-steps__item") # Extrair
preparação (lista)

preparacao = [item.get_text(strip=True) for item in elementos_preparacao] if
elementos_preparacao else ["Preparação não encontrada"]

nome_arquivo = f"{titulo.replace(" ", "_").txt}" # Salvar num arquivo .txt

caminho_arquivo = os.path.join(pasta_destino, nome_arquivo)

with open(caminho_arquivo, "w", encoding="utf-8") as f:

    f.write(f"{titulo}\n")

    f.write(f"Receita de: {autor}\n")

    f.write(f"\n{titulo_ingredientes}\n")

    for ingrediente in ingredientes:

```



```

        f.write(f"{ingrediente}\n")

    f.write(f"\n{titulo_preparacao}\n")

    for passo in preparacao:

        f.write(f"{passo}\n")

print(f"Texto extraído com sucesso e salvo em: {caminho_arquivo}")

except requests.exceptions.RequestException as e:

    print(f"Erro ao encontrar a URL: {e}")

```

Explicação do código:

A função **extrair_texto_de_url** extrai informações de receitas de uma página do site **24 Kitchen** e guarda-as num arquivo .txt. Esta função utiliza a biblioteca *requests* para fazer a requisição HTTP, verificando possíveis erros com o comando `raise_for_status()`. O conteúdo da página é analisado com o BeautifulSoup para extrair o título da receita, o seu autor, os seus ingredientes e as suas etapas de preparação. Os elementos HTML específicos, tais como as classes e as tags, são usados para identificar as secções de interesse. Posteriormente, os dados obtidos são organizados num arquivo de texto (.txt), com o respectivo nome baseado no título da receita e formatados em secções bem definidas. Por último, a função também possui tratamento de erros, sendo capaz de exibir mensagens claras em caso de falha na conexão ou na extração dos dados.

Tarefa 3

Para a terceira tarefa, era exigido amplificar o trabalho desenvolvido na alínea anterior para que o código python tivesse incorporadas ferramentas de análise textual, tais como:

- Calcular o número de palavras/linhas no texto;
- Calcular a frequência de uma palavra no texto;
- Determinar as 10 palavras mais comuns no texto.
- Encontrar todos os nomes próprios no texto;
- Calcular o comprimento médio das frases do texto;
- Encontrar todas as palavras que os textos possuem em comum;

Para tal, desenvolvemos os seguintes códigos:

O primeiro código realiza as primeiras 5 subtarefas, enquanto que o segundo código calcula a última sub tarefa.

```
# Função para analisar o texto dos arquivos .txt

def analise_texto(arquivo):

    with open(arquivo, "r", encoding="utf-8") as f:

        texto = f.read()

        palavras = re.findall(r"\b[\w./]+\b", texto.lower()) # Inclui números fracionários como
        "1/2" e palavras como "q.b."

        linhas = texto.splitlines()

        palavra_alvo = input("Digite uma palavra para calcular a frequência: ").strip().lower() #
        3. Frequência de uma palavra a escolher no texto

        freq_palavra = palavras.count(palavra_alvo)

        num_palavras = len(palavras) # 2. Calcula o número de palavras no texto

        num_linhas = len(linhas) # 3. Calcula o número de linhas no texto

        nomes_proprios = re.findall(r"(?<=Receita de: )([A-Za-z ]+)", texto) # 4. Encontra os nomes
        próprios no texto, ou seja, o que está depois de "Receita de:"

        frases = re.split(r"[.!?]", texto) # 5. Calcula o comprimento médio das frases no texto

        frases_limpa = [frase.strip() for frase in frases if frase.strip()]

        comprimento_medio_frases = sum(len(frase.split()) for frase in frases_limpa) /
        len(frases_limpa) if frases_limpa else 0

        frequencias = Counter(palavras) # 6. Calcula as 10 palavras mais comuns no texto

        palavras_mais_comuns = frequencias.most_common(10)

    return {

        "arquivo": arquivo,

        "freq_palavra": freq_palavra,

        "num_palavras": num_palavras,

        "num_linhas": num_linhas,
```

```

"nomes_proprios": nomes_proprios,

"comprimento_medio_frases": comprimento_medio_frases,

"palavras_mais_comuns": palavras_mais_comuns,

}

```

```

# Função para encontrar palavras comuns entre textos, excluindo palavras simples

def analisar_palavras_comuns(caminho_pasta):

    palavras_simples = {"não", "e", "ou", "o", "a", "os", "as", "um", "uma", "de", "da",
                        "do", "em", "no", "na", "para",

                        "com", "por", "como", "quando", "onde", "que", "se", "mas"}

    conjuntos_palavras = []

    for arquivo in os.listdir(caminho_pasta):

        if arquivo.endswith(".txt"):

            caminho_arquivo = os.path.join(caminho_pasta, arquivo)

            with open(caminho_arquivo, "r", encoding="utf-8") as f:

                texto = f.read().lower()

                palavras = re.findall(r"\b[\w.]+\b", texto) # Inclui palavras como "q.b."

                palavras_filtradas = {palavra for palavra in palavras if palavra not in
palavras_simples and not palavra.isdigit()} # Filtra as palavras, mantendo apenas as que não
estão em 'palavras_simples'

                conjuntos_palavras.append(palavras_filtradas)

    if conjuntos_palavras:

        return set.intersection(*conjuntos_palavras) # Palavras comuns entre os textos

    return set()

```

Explicação do código:

A função **analise_texto** realiza uma análise detalhada do conteúdo de um arquivo .txt. Começa por ler o texto completo do arquivo e processa-o de modo a extrair diversas informações. Entre os dados analisados estão: **o número total de palavras e linhas, a frequência de uma palavra específica fornecida pelo utilizador, os nomes próprios** (encontrados após a expressão "Receita de:"), **o comprimento médio das frases e as 10 palavras mais comuns no texto**. Para isso, a função utiliza expressões regulares, divisão de texto em linhas e frases, além de ferramentas como o counter para calcular a frequência de palavras. Os resultados são organizados num dicionário, tornando a função útil para compreender e resumir o conteúdo textual de arquivos de receitas ou outros documentos formatados de maneira semelhante.

A função **analisar_palavras_comuns** identifica palavras em comum entre os arquivos de texto armazenados numa pasta, excluindo palavras simples e números. Inicialmente, definimos um conjunto de palavras consideradas simples, como "e", "de", "que", entre outras. A função percorre todos os arquivos .txt no diretório especificado, lendo o conteúdo de cada um e convertendo-o para minúsculas. As palavras são extraídas utilizando uma expressão regular e somente as que não pertencem ao conjunto de palavras simples ou que não sejam números são mantidas. Em seguida, os conjuntos de palavras filtradas de cada arquivo são armazenados numa lista. Por fim, a intersecção desses conjuntos é calculada, devolvendo as palavras que aparecem em todos os arquivos analisados. Caso nenhum arquivo seja processado, a função retorna um conjunto vazio.

Tarefa 4

Por último, para a quarta tarefa, tínhamos por mãos criar uma interface para interagir com o programa, neste caso uma janela do terminal na qual fosse possível consultar e calcular todos os valores apresentados na tarefa anterior (tarefa 3). Também era exigido que fosse possível passar os dados de cada ficheiro para um formato adequado à escolha de quem executasse o programa (JSON, XML, CSV).

Para tal, desenvolvemos o seguinte código:

O primeiro código é relativo à exportação para certos tipos de ficheiros, enquanto que o segundo código refere-se ao menu para a interface.

```
# Função para exportar os resultados para ficheiros json, csv ou xml  
  
def exportar_resultados(resultados, formato):
```

```

    diretorio_destino =
r"C:\Users\joana\Desktop\Joana\Universidade\UMinho\HD\FPLN\Trabalho_Final\receitas" #
Definir o diretório "dados exportados" no caminho desejado

    if not os.path.exists(diretorio_destino): # Criar o diretório caso não exista

        os.makedirs(diretorio_destino)

    nome_base = os.path.splitext(os.path.basename(resultados["arquivo"]))[0] # Nome base do
arquivo (sem extensão)

    if formato == "json":

        with open(f"{diretorio_destino}/{nome_base}.json", "w") as f: # Exportar para o
formato escolhido

            json.dump(resultados, f, indent=4)

            print(f"Resultados exportados para {diretorio_destino}/{nome_base}.json")

    elif formato == "csv":

        with open(f"{diretorio_destino}/{nome_base}.csv", "w", newline="") as f:

            writer = csv.writer(f)

            writer.writerow(resultados.keys())

            writer.writerow(resultados.values())

            print(f"Resultados exportados para {diretorio_destino}/{nome_base}.csv")

    elif formato == "xml":

        root = ET.Element("Resultados")

        for key, value in resultados.items():

            child = ET.SubElement(root, key)

            child.text = str(value)

        tree = ET.ElementTree(root)

        tree.write(f"{diretorio_destino}/{nome_base}.xml")

```

```

        print(f"Resultados exportados para {diretorio_destino}/{nome_base}.xml")

    else:

        print("Formato inválido. Exportação não realizada.")

```

```

def main():
    while True:
        print("\nEscolha uma opção:")
        print("1. Ler os arquivos .txt na pasta.")
        print("2. Extrair as informações da URL do 24Kitchen.")
        print("3. Contar a frequência de uma palavra à escolha e analisar os arquivos .txt na pasta.")
        print("4. Encontrar as palavras comuns entre os arquivos .txt.")
        print("5. Sair")

        escolha = input("Opção: ").strip()

        if escolha == "1":
            caminho_pasta = r"C:\Users\joana\Desktop\Joana\Universidade\UMinho\HD\FPLN\Trabalho_Final\receitas"
            if not os.path.exists(caminho_pasta):
                print("Caminho não encontrado. Verifique e tente novamente.")
                continue

            arquivos_txt = [arq for arq in os.listdir(caminho_pasta) if arq.endswith(".txt")]
            if not arquivos_txt:
                print("A pasta não contém arquivos .txt.")
                continue

            print("\nArquivos disponíveis:")
            for i, arquivo in enumerate(arquivos_txt, start=1):
                print(f"{i}. {arquivo}")

            escolha_arquivo = input("\nEscolha o número do arquivo para analisar: ").strip()
            if not escolha_arquivo.isdigit() or int(escolha_arquivo) < 1 or int(escolha_arquivo) > len(arquivos_txt):
                print("Escolha inválida. Tente novamente.")

            caminho_arquivo = os.path.join(caminho_pasta, arquivos_txt[int(escolha_arquivo) - 1])
            print(f"\nAnalisando o arquivo: {arquivos_txt[int(escolha_arquivo) - 1]}")

            # Chama a função para extrair as informações do arquivo selecionado
            resultado = extrair_informacoes_pasta(caminho_pasta)[int(escolha_arquivo) - 1]

            # Exibir as informações do arquivo selecionado

```

```

        print(f"Titulo: {resultado['titulo']}")
        print(f"Autor: {resultado['autor']}")
        print("\nIngredientes")
        for ingrediente in resultado["ingredientes"]:
            print(f"{ingrediente}")
        print("\nPreparação")
        for passo in resultado["preparacao"]:
            print(f"{passo}")

    elif escolha == "2":
        url = input("Insere a URL da receita do 24Kitchen: ").strip()
        pasta_destino = "./receitas_24kitchen"
        os.makedirs(pasta_destino, exist_ok=True)
        extrair_texto_de_url(url, pasta_destino)

    elif escolha == "3":
        caminho_pasta =
r"C:\Users\joana\Desktop\Joana\Universidade\UMinho\HD\FPLN\Trabalho_Final\receitas"
        if not os.path.exists(caminho_pasta):
            print("Caminho não encontrado. Verifique e tente novamente.")
            continue

        arquivos_txt = [arq for arq in os.listdir(caminho_pasta) if
arq.endswith(".txt")]
        if not arquivos_txt:
            print("A pasta não contém arquivos .txt.")
            continue

        print("\nArquivos disponíveis:")
        for i, arquivo in enumerate(arquivos_txt, start=1):
            print(f"{i}. {arquivo}")

        escolha_arquivo = input("\nEscolha o número do arquivo para analisar:
").strip()
        if not escolha_arquivo.isdigit() or int(escolha_arquivo) < 1 or
int(escolha_arquivo) > len(arquivos_txt):
            print("Escolha inválida. Tente novamente.")
            continue

        caminho_arquivo = os.path.join(caminho_pasta, arquivos_txt[int(escolha_arquivo)
- 1])

        print(f"\nAnalisando o arquivo: {arquivos_txt[int(escolha_arquivo) - 1]}")
        resultados = analise_texto(caminho_arquivo)

        print(f"Frequência da palavra escolhida: {resultados["freq_palavra"]}")

        print(f"\nNúmero de palavras: {resultados["num_palavras"]}")

        print(f"\nNúmero de linhas: {resultados["num_linhas"]}")

        print("Nomes próprios encontrados:", end=" ")
        print(", ".join(resultados["nomes_proprios"]) if resultados["nomes_proprios"]
else "Nenhum nome próprio encontrado.")

        print(f"Comprimento médio das frases:
{resultados["comprimento_medio_frases"]:.2f} palavras.")

```

```

        print("\n10 palavras mais comuns:")
        for palavra, freq in resultados["palavras_mais_comuns"]:
            print(f" - {palavra}: {freq}")

        # Exportar resultados
        exportar = input("\nDeseja exportar os resultados? (Sim/Não): ")
        exportar = exportar.strip().lower()

        if exportar == "Sim":
            formato = input("Escolha o formato (json/csv/xml): ").strip().lower()
            if formato in ["json", "csv", "xml"]:
                exportar_resultados(resultados, formato)
            else:
                print("Formato inválido. Exportação não realizada.")
        else:
            print("Exportação ignorada.")

        elif escolha == "4":
            caminho_pasta = r"C:\Users\joana\Desktop\Joana\Universidade\UMinho\HD\FPLN\Trabalho_Final\receitas"

            if not os.path.exists(caminho_pasta):
                print("Caminho não encontrado. Verifique e tente novamente.")
                continue

            arquivos_txt = [arq for arq in os.listdir(caminho_pasta) if
arq.endswith(".txt")]# Verifica se existem arquivos .txt na pasta
            if not arquivos_txt:
                print("A pasta não contém arquivos .txt.")
                continue

            palavras_comum = analisar_palavras_comuns(caminho_pasta)# Chama a função para
encontrar palavras comuns

            if palavras_comum:
                print("\nPalavras comuns entre os arquivos:")
                for palavra in palavras_comum:
                    print(f" - {palavra}")
            else:
                print("\nNão foram encontradas palavras comuns entre os arquivos.")

        elif escolha == "5":
            print("Programa encerrado. Até breve!")
            break

        else:
            print("Opção inválida. Tente novamente.")

if __name__ == "__main__":
    main()

```


Explicação do código:

A função **exportar_resultados** permite exportar os resultados de análises ou extrações para os formatos **JSON**, **CSV** ou **XML**, armazenando-os num diretório específico no computador. Inicialmente, verifica-se se o diretório de destino existe, criando-o caso necessário. O nome base do arquivo é gerado a partir do nome do arquivo original presente no dicionário de resultados. Dependendo do formato escolhido pelo utilizador, a função realiza as seguintes operações:

- **JSON:** Converte os dados do dicionário para o formato JSON e guarda-os com indentação para facilitar a leitura.
- **CSV:** Grava os dados do dicionário como uma tabela, com as chaves como cabeçalhos e os valores numa única linha.
- **XML:** Cria uma estrutura XML, onde cada chave do dicionário se torna um elemento com o seu respectivo valor como texto.

Por fim, o código exibe uma mensagem no output indicando o sucesso da exportação ou informa se o formato especificado é inválido.

A função **main** implementa o menu principal de um programa interativo para manipular e analisar os dados extraídos de arquivos de texto e páginas da web. O menu apresenta cinco opções principais:

1. **Ler arquivos .txt na pasta:**
Elenca os arquivos .txt disponíveis numa pasta específica e permite ao utilizador selecionar um para exibir as informações detalhadas, como **o título, o autor, os ingredientes e a preparação**.
2. **Extrair informações de uma URL do 24Kitchen:**
Permite ao utilizador inserir um URL da receita do site 24 Kitchen, extrair as suas informações e gravá-las num arquivo .txt no diretório pré-determinado.
3. **Analisar arquivos .txt:**
Realiza a análise de um arquivo de texto selecionado, calculando métricas como a frequência de uma palavra escolhida, o número de palavras e linhas, nomes próprios encontrados, comprimento médio das frases e as 10 palavras mais comuns. Opcionalmente, o utilizador pode exportar os resultados em formatos como **JSON, CSV ou XML**.
4. **Encontrar palavras comuns entre arquivos:**
Identifica palavras que aparecem em todos os arquivos .txt da pasta, excluindo palavras simples e números.

5. **Sair:**

Encerra o programa.

A função também inclui validações para garantir que os caminhos, arquivos e opções fornecidos pelo utilizador sejam válidos, além de mensagens de erro claras para situações como pastas inexistentes ou arquivos ausentes.

Conclusões

Em conclusão, apreciamos a realização deste projeto, uma vez que possibilitou conciliar os conhecimentos adquiridos em contexto de aula, assim como aplicá-lo de forma prática num contexto por nós admirado, como é o caso da vertente da culinária, de onde extraímos receitas sob o formato .txt e posteriormente as analisamos à luz desses mesmos conhecimentos adquiridos em aula. Apesar de desafiante, concluímos que a linguagem python é extremamente útil para extrair informação de forma concisa, rápida e eficaz de um website, ao invés de procurar informação manualmente de um dado website. Achamos também uma vantagem assentarmos a matéria e a sua respectiva utilidade de uma forma mais natural e interativa ao realizar o trabalho.

Bibliografia

“Hashável”

PHP.net. (n.d.). *Classe Ds\Hashable*. Acessado a 18 de janeiro de 2025, de https://www.php.net/manual/pt_BR/class.ds-hashable.php

APIs

Amazon Web Services. (n.d.). *O que é uma API?* Acessado a 18 de janeiro de 2025, de <https://aws.amazon.com/pt/what-is/api/>

CSV

Google. (n.d.). *Sobre anúncios responsivos de display*. Acessado a 18 de janeiro de 2025, de <https://support.google.com/google-ads/answer/9004364?hl=pt>

JSON

Hostinger. (n.d.). *O que é JSON? Tutorial completo para iniciantes*. Acessado a 18 de janeiro de 2025, de <https://www.hostinger.pt/tutoriais/o-que-e-json>

HTTP

Akamai Technologies. (n.d.). *O que é HTTP?* Recuperado em 18 de janeiro de 2025, de <https://www.akamai.com/pt/glossary/what-is-http>

XML

Amazon Web Services. (n.d.). *O que é XML?* Recuperado em 18 de janeiro de 2025, de <https://aws.amazon.com/pt/what-is/xml/>